CSE 144 Final Project Report

Transfer Learning Challenge

Group Members
Tristan Nguyen (tnguy559, 1929743)

Spring 2026

1. Introduction

The goal of this project is to classify images into 100 distinct categories using a machine learning model trained on a limited dataset of approximately 10 labeled images per class.

Transfer learning is highly appropriate for this problem because the training dataset is far too small to train a deep neural network from scratch. Doing so would result in severe overfitting and poor generalization. Instead, a model pretrained on ImageNet already has rich visual feature representations that can be adapted to the target 100 classes with minimal data.

The approach fine-tunes a pretrained ResNet-50 backbone by unfreezing the last two residual blocks (layer3 and layer4) and replacing the final fully connected layer with a new classifier head. The best submission achieved 60.9% test accuracy on the Kaggle public leaderboard, exceeding the required 60% baseline.

2. Dataset

The dataset consists of 100 classes sampled from multiple source datasets. Each class contains approximately 10 labeled training images (1079 total), split 80/20 into 864 training and 215 validation images. The test set contains 1036 unlabeled images for Kaggle submission.

The training directory is structured with 100 subdirectories named 0 through 99, each containing the images for that class. PyTorch's ImageFolder loader is used to load images, with a critical correction applied in which the default alphabetical sorting of folder names produces incorrect label assignments.

All images are resized to 224x224 pixels to match ResNet-50's expected input dimensions. Pixel values are normalized using ImageNet statistics to match the distribution the pretrained model was trained on. Training images additionally have random horizontal flipping and random cropping with padding to increase diversity. Validation images receive only resizing and normalization.

3. Implementation

ResNet-50 pretrained on ImageNet is used as the backbone, loaded with TorchVision's models.resnet50. ResNet-50 was chosen because its ~25 million parameters strike a balance between representational capacity and overfitting risk for a small dataset. It is well known as a strong transfer learning baseline for image classification tasks.

The original fully connected layer is replaced with a custom classifier head consisting of Dropout(p=0.3) followed by Linear(2048, 100). The dropout layer acts as regularization to reduce overfitting on the small training set. This results in 22,268,004 trainable parameters.

All layers are initially frozen to preserve the pretrained ImageNet weights. The last two residual blocks (layer3 and layer4) are then selectively unfrozen to allow fine-tuning of higher-level visual features relevant to the target classes, while earlier layers detecting basic features like edges or textures stay frozen.

Cross Entropy Loss is used as the loss function, which is standard for multi-class classification. The AdamW optimizer is used with differential learning rates: 1e-5 for the unfrozen pretrained layers (layer3 and layer4) and 1e-4 for the new classifier head. The smaller learning rate for pretrained layers prevents overwriting the valuable ImageNet weights. Training runs for 30 epochs with a batch size of 32.

4. Experiments

The baseline setup freezes all layers and only trains the new FC head, giving approximately 36% validation accuracy. This confirms that the pretrained features alone are insufficient without any task-specific adaptation.

Hyperparameter tuning focused on how many layers to unfreeze. Unfreezing only layer4 improved validation accuracy to ~56%. Unfreezing both layer3 and layer4 further improved it to ~65%. The 80/20 validation split with fixed seed 42 is used consistently across all experiments.

Several fixes were explored during development. Unfreezing more layers consistently improved accuracy, with layer3 and layer4 together yielding the best results. Adding Dropout(0.3) before the FC layer reduced overfitting given the large gap between training and validation accuracy. Training on the full dataset without a validation split hurt Kaggle performance (58.2%) since overfitting went undetected and the final checkpoint was used instead of the best one.

5. Results

The best model achieves 65.58% validation accuracy at epoch 23 out of 30, with training accuracy reaching ~98%. The gap between training and validation accuracy is consistent with mild overfitting expected on a dataset of this size. The Kaggle public leaderboard score for the best submission is 60.909%, exceeding the required 60% baseline.

Training loss decreases consistently from 4.5 at epoch 1 to 0.3 at epoch 30. Validation loss decreases from 4.4 to 1.4, with validation accuracy improving most rapidly in the first 15 epochs before plateauing.

6. Discussion

The most impactful decision was unfreezing layer3 and layer4, which allowed the higher-level feature extractors to adapt to the target domain. The differential learning rate strategy was essential in which using a uniform learning rate would destroy the pretrained weights. The label ordering fix was also important as an early submission without it scored only 16.4% despite 65% validation accuracy.

The primary failure is overfitting in which training accuracy consistently exceeded validation accuracy by 30+ percentage points in later epochs. This is expected given only 8-9 training images per class after the 80/20 split. Removing the validation split to train on all data made overfitting undetectable and made a worse Kaggle score. Key limitations include the extremely small per-class sample size and the inability to monitor generalization without sacrificing training data.

7. Reproducibility

Random seed 42 is set globally for Python's random module, NumPy, PyTorch CPU, and PyTorch CUDA through a set_seed(42) function called at the start of execution. torch.backends.cudnn.deterministic=True and torch.use_deterministic_algorithms(True) are

enabled. The train/val split uses torch.Generator().manual_seed(42) to ensure the same split across runs.

Environment was Python 3.12.12, PyTorch 2.x, TorchVision (matching version), NumPy, Pandas, Matplotlib, tqdm. All dependencies are pre-installed in Kaggle Notebooks with no additional pip installs required.

8. Team Contributions

Tristan Nguyen – dataset preparation, model implementation, training, evaluation, and report

9. References

1. TorchVision Models Documentation. https://pytorch.org/vision/stable/models.html
2. PyTorch Documentation. https://pytorch.org/docs/stable/index.html
3. Kaggle Competition: UCSC CSE 144 Spring 2026 Final Project. https://www.kaggle.com/competitions/ucsc-cse-144-spring-2026-final-project